

ResumeIQ – AI-Based Resume Screening and Job Matching System

A PROJECT BASED LEARNING (PBL) REPORT

Submitted by

SADHANA SHANMUGAM - 2104251040837

DHANYA SRI MOHANDASS - 2104251040189

Submitted in partial fulfilment of the requirements

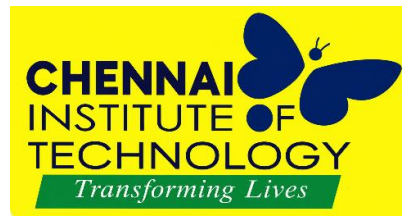
for the

Project-Based Learning component of Machine Learning

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI



Affiliated to Anna University, Chennai

(Autonomous)

OCTOBER 2026

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

IM1. To create next generation leaders by effective teaching learning methodologies and in still Scientifics Park in them to meet the global challenges.

IM2. To transform lives through deployment of emerging technology, novelty and sustainability.

IM3. To inculcate human values and ethical principles to cater the societal needs.

IM4. To contribute towards the research ecosystem by providing a suitable,

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

DM1. To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.

DM2. To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.

DM3. To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.

BONAFIDE CERTIFICATE

This is to certify that the Project-Based Learning report titled **“ResumeIQ – AI-Based Resume Screening and Job Matching System”** is a Bonafide record of work carried out by **Sadhana Shanmugam [2104251040837]** , **Dhanya Sri Mohandass [2104251040189]** of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor-guided Project-Based Learning (PBL) component of the Machine Learning course during the academic year [2026–2027] under my supervision.

SIGNATURE

Dr. S. PAVITHRA, M.E., Ph.D.,
Professor and Head,
Dept. of Computer Science and Engineering
Chennai Institute of Technology,
Chennai – 69.

SIGNATURE

Mrs. POORNIMA LAKSHMI,
MENTOR
Assistant Professor
Dept. of Computer Science and Engineering
Chennai Institute of Technology,
Chennai – 69.

Submitted for the final review held on

Internal Examiner

External Examiner

DECLARATION

I/We jointly declare that the PBL report on **“ResumeIQ – AI-Based Resume Screening and Job Matching System”** is the result of original work done by us and best of our knowledge, similar work has not been submitted to **“ANNA UNIVERSITY, CHENNAI”** for the requirement of Degree of **BACHELOR OF ENGINEERING**. This PBL report is submitted on the partial fulfilment of the requirement of the award of Degree of **COMPUTER SCIENCE AND ENGINEERING**.



Signature

Sadhana Shanmugam

Dhanya Sri Mohandass

Place: Chennai

Date:

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honourable Chairman **SHRI. P. SRIRAM** for providing immense facilities at our institution.

We are very proud to render our thanks to our Principal **Dr. A. RAMESH M.E., Ph.D.**, for the facilities and the encouragement given by him toward the progress and completion of our project.

We would like to express special thanks and gratitude to our Dean **Dr. V. SRINIVASA RAO M.E., Ph.D.**, who has been a key source of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr. S. PAVITHRA M.E., Ph.D.**, for her effective leadership, encouragement and guidance throughout the project.

We would like to extend our thanks to the Project Co-ordinator **Mrs. POORNIMA LAKSHMI**, Assistant Professor, Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from our class advisors **Dr. G. IRIN LORETTA M.E.**, Assistant Professor, and **S.E. NEELA KANDAN M.TECH.**, Assistant Professor, of the Department of Computer Science and Engineering for their valuable suggestions and support toward the successful completion of the project.

Sadhana Shanmugam (2104251040837)

Dhanya Sri Mohandass (2104251040189)

ABSTRACT

Recruiters routinely receive hundreds of resumes for a single vacancy, and screening them by hand is slow, inconsistent and prone to human error and bias, so qualified candidates can be overlooked. ResumeIQ is an AI-based resume analysis and job-matching system that automates this first screening stage and lets a recruiter manage the outcome. Resumes in PDF, DOC, DOCX and TXT formats are converted to text, and a rule-based natural-language-processing layer extracts skills (from a 164-skill taxonomy), education level, years of experience and contact details. Names, e-mail addresses and phone numbers are removed before scoring so that they cannot influence a ranking. The cleaned text is converted into TF-IDF vectors (unigrams and bigrams, 11,688 features), classified into one of 12 job roles by a Multinomial Naive Bayes model and a calibrated linear Support Vector Machine whose probabilities are averaged, and compared with a job description by cosine similarity. A transparent five-part match score (text similarity 40%, skill coverage 30%, role fit 15%, experience 10% and education 5%) ranks the candidates and recommends a top-N shortlist with matched, related and missing skills for every candidate. The system is delivered as a multi-user Flask web application: recruiters register and log in, every screening is saved in a relational database (SQLite for development, PostgreSQL in production, with Alembic migrations), and candidates can be shortlisted, rejected, annotated, searched and exported to CSV. Passwords are hashed with scrypt, forms are CSRF-protected, login attempts are rate-limited, and all secrets are read from environment variables. The application is packaged with Docker and a Render blueprint and is covered by 78 automated tests that run on both databases. The models were trained on 2,400 synthetic resumes (12 roles $\times$ 200) because real resumes are personal data. On a held-out test set of 480 resumes the ensemble reached 99.8% accuracy (479/480), with 5-fold cross-validation accuracy of 99.7% (Naive Bayes) and 99.4% (SVM); all five hand-written resumes that the data generator never produced were matched to the correct role, and 14 sample resumes in four file formats were screened and saved in a median of 0.19 s. Because the training data are synthetic, these figures show that the pipeline works and are not an estimate of real-world accuracy; the report states this limitation openly and documents a code-ready path to retrain on real labelled resumes.

Keywords: Resume Screening, Job Matching, Natural Language Processing, TF-IDF, Naive Bayes, Support Vector Machine.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Driving Question	1
1.3 Objectives	2
1.4 Scope and Limitations	2
CHAPTER 2: CONCEPT EXPLORATION	3
2.1 Related Approaches	3
2.2 Summary Table	4
2.3 What This Told Us	4
CHAPTER 3: PROJECT PLANNING AND TEAM ORGANISATION	5
3.1 Weekly PBL Progress Log	5
3.2 Requirements	5
3.3 Feasibility	6
CHAPTER 4: ITERATIVE DESIGN AND DEVELOPMENT	7
4.1 System Architecture	7
4.2 Iteration 1: Baseline	8
4.3 Iteration 2: Refinement	8
4.4 Final Approach	9
4.5 Training Procedure	10
4.6 Iteration 3: Multi-user Platform	11
CHAPTER 5: IMPLEMENTATION	13
5.1 Module Description	13
5.2 Key Code Snippets	14
5.3 User Interface / Demo	14
5.4 Interfaces and Deployment	17
CHAPTER 6: RESULTS AND DISCUSSION	19
6.1 Evaluation Metrics	19
6.2 Results Across Iterations	19
6.3 Platform Verification and Performance	22
6.4 Discussion	22
6.5 Limitations	23
CHAPTER 7: TEAM REFLECTION AND LEARNING OUTCOMES	24
7.1 Individual Reflections	24
7.2 Team Learning	24
7.3 Course Outcomes — Evidence Summary	25
CHAPTER 8: CONCLUSION AND FUTURE SCOPE	26
8.1 Conclusion	26
8.2 Future Scope	26
REFERENCES	27
APPENDIX	28
A.1 Full Source Code	28
A.2 Complete Weekly PBL Log	28
A.3 Self and Peer Assessment	28

LIST OF FIGURES

Figure 4.1 System Architecture Diagram	7
Figure 4.2 ML Scoring Pipeline Data Flow	10
Figure 5.1 Dashboard After Logging In	15
Figure 5.2 New Screening Page with a Job Description and Resumes	15
Figure 5.3 Saved Screening: 15 Resumes Ranked, AI Top 5 Shortlisted	16
Figure 5.4 Shortlist Page Across All Screenings	16
Figure 6.1 Evaluation Metrics: Final Model and Accuracy Across Data Iterations	19
Figure 6.2 Confusion Matrix of the Ensemble on the Hold-out Set	20
Figure 6.3 Live Ranking of the Sample Resumes for the Data Scientist Job	21
Figure 7.1 Course Outcomes	25



LIST OF TABLES

Table 2.1 Summary of Related Approaches	4
Table 3.1 Weekly PBL Progress Log	5
Table 3.2 Hardware and Software Requirements	6
Table 4.1 Components of the Match Score	10
Table 4.2 Dataset and Model Configuration	11
Table 4.3 Database Tables (SQLAlchemy models, created by an Alembic migration)	11
Table 4.4 Security Controls	12
Table 5.1 Web Pages and JSON API Endpoints	17
Table 5.2 Automated Test Suite (78 tests)	18
Table 6.1 Model Evaluation Results Across Iterations	20
Table 6.2 Per-role Results of the Ensemble on the Hold-out Set	21
Table 6.3 Top Candidates for the Data Scientist Job (14 Sample Resumes)	21
Table 6.4 Measured Performance and Verification of the Platform	22



LIST OF ABBREVIATIONS

Abbreviation	Full Form
ML	Machine Learning
NLP	Natural Language Processing
TF-IDF	Term Frequency – Inverse Document Frequency
NB	Naive Bayes
SVM	Support Vector Machine
CV	Cross-Validation
PBL	Project-Based Learning
PII	Personally Identifiable Information
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
CSV	Comma-Separated Values
PDF	Portable Document Format
DOCX	Office Open XML Word Document
OCR	Optical Character Recognition
NER	Named Entity Recognition
CSP	Content Security Policy
CI	Continuous Integration
UI	User Interface
WSGI	Web Server Gateway Interface
ORM	Object-Relational Mapping
CSRF	Cross-Site Request Forgery
HSTS	HTTP Strict Transport Security
SQL	Structured Query Language

TEAM ROLES AND RESPONSIBILITIES

Both members jointly owned problem framing, the weekly mentor-review cycle, the literature exploration in Chapter 2, and the final report, and contributed **equally (50% : 50%)** to the project. Division of the build work below reflects primary ownership; both members reviewed and tested the full system before each milestone.

Team Member	Primary Role	Key Responsibilities
Sadhana Shanmugam (2104251040837)	ML Pipeline & Data Backend Lead	Designed and implemented the resume text extraction (extractor.py), the rule-based NLP parser and skill taxonomy (parser.py, skills.py), the TF-IDF / Naive Bayes / SVM training and evaluation pipeline (train.py, dataset.py), the scoring and role-recommendation engine (engine.py), and, for the web platform, the SQLAlchemy data model, Alembic migrations, the screening and shortlist JSON API and the environment-driven configuration.
Dhanya Sri Mohandass (2104251040189)	Interface, Security, Testing & Deployment Lead	Designed and implemented the web interface (dashboard, screening, shortlist, account and model-insight pages in HTML, CSS and JavaScript), the account system (registration, login, logout, CSRF protection, rate limits and security headers), the sample resumes and screenshots, the 78-test automated suite, and the Docker image, Render blueprint and CI workflow.

CHAPTER 1

INTRODUCTION

1.1 Background

Hiring begins with a screening problem. A single vacancy can attract hundreds or thousands of applications, and recruiters must decide, often in minutes per resume, which deserve an interview. Done by hand, this first pass is slow, tiring and inconsistent: two reviewers may rank the same resume differently, and a qualified candidate can be overlooked simply because the resume words a skill differently from the advertisement. Keyword filters reduce the workload but share the weakness, because “machine learning” and “ML”, or “scikit-learn” and “sklearn”, are different strings to a filter but the same skill to a person.

Resumes also arrive as unstructured documents (PDF, Word, plain text) with no common layout, so the text must first be extracted and facts such as skills, education and years of experience recovered from free text. A text-classification model can then learn which vocabulary characterises each job family, and similarity over weighted term vectors can quantify how closely a resume matches a job description.

Automation is not automatically fair. A review of algorithmic-hiring practice found that vendors say little about how bias is validated [1], and in 2018 a company withdrew an experimental recruiting tool found to disadvantage women [2]. A screening system should therefore be explainable, keep personal identifiers out of its scoring, and be positioned as a decision aid rather than a decision maker.

ResumeIQ was undertaken as the Project-Based Learning (PBL) component of the Machine Learning course (CS5305), following the build–learn cycle that PBL is meant to teach: a baseline text classifier, a refinement driven by what the first results revealed, and a final system the team can explain component by component.

1.2 Driving Question

Can a classical machine-learning pipeline built from TF-IDF features, Naive Bayes and Support Vector Machine classifiers, and cosine similarity turn unstructured resumes into a ranked, explainable shortlist for a given job description, and recommend suitable job roles for a single resume, without using the candidate's name or contact details and while running on ordinary laptop hardware? And if two structurally different classifiers are used, does combining them give a more reliable role signal than either one alone?

This narrows into a buildable task: extract skills, education and experience with NLP; represent resume and job text as TF-IDF vectors; classify each resume into one of twelve roles with Naive Bayes and an SVM; and combine text similarity, skill coverage, role agreement, experience and education into one score a recruiter can inspect.

1.3 Objectives

- To accept resumes in PDF and DOC formats (and DOCX and plain text) and convert them reliably into text.
- To extract skills, education and experience using NLP, removing names and contact details before scoring.
- To match resumes with job descriptions and measure suitability using TF-IDF vectors and cosine similarity.
- To apply TF-IDF, Naive Bayes and SVM for ranking and classification, evaluated with a held-out set and cross-validation.
- To generate a top-candidate shortlist automatically, explaining each candidate with matched, related and missing skills, and to recommend suitable roles for a single resume.
- To deliver a working multi-user website built with Python, Scikit-learn and Pandas, with accounts, saved screenings and shortlist management on a relational database, packaged for deployment on Render, and to document the weekly PBL progress and what each member learned.

1.4 Scope and Limitations

ResumeIQ models 12 technology and business job roles (for example Data Scientist, DevOps Engineer, QA Engineer and Business Analyst) and works on English, text-based resumes. Skills come from a taxonomy of 164 skills with aliases (for example “sklearn” for Scikit-learn), so skills outside it are not detected. Scanned, image-only documents have no text layer and are rejected with a clear message; OCR is outside the scope of this build.

The classifiers were trained on 2,400 synthetic resumes, because real resumes are personal data and no real labelled corpus was available. The reported accuracy therefore measures how well the pipeline separates the synthetic roles and is not a real-world estimate (Chapter 6). Resume files are processed in memory and never stored; only the extracted details and scores are saved to the recruiter's own account, and every shortlist should be reviewed by a person.

CHAPTER 2

CONCEPT EXPLORATION

2.1 Related Approaches

2.1.1 Text representation and similarity

Spärck Jones proposed in 1972 that terms occurring in fewer documents are better discriminators and should be weighted by their inverse document frequency [3]. Salton and Buckley compared many term-weighting schemes and found that those combining term frequency, inverse document frequency and length normalisation performed well [4]. In the resulting TF-IDF representation each document is a sparse vector, and after length normalisation the cosine of the angle between two vectors measures how much weighted vocabulary the documents share [5]. This is what is needed to compare a resume with a job description, and it needs no labelled data, so it forms the similarity component of ResumeIQ's score.

2.1.2 Probabilistic text classification

McCallum and Nigam compared two Naive Bayes event models and found that the multinomial model, which uses word counts, generally performed better than the Bernoulli model when the vocabulary was large [6]. Naive Bayes trains in a single pass, works with little data and gives class probabilities directly, so it was chosen as the first classifier; its main weakness is the independence assumption, which makes its probabilities over-confident.

2.1.3 Support Vector Machines for text

Cortes and Vapnik introduced the soft-margin support-vector network [7]. Joachims argued that text suits SVMs because documents are high-dimensional, sparse and mostly linearly separable, and reported that SVMs outperformed k-nearest-neighbour, Naive Bayes, Rocchio and decision-tree classifiers in his experiments [8]. Linear SVMs for sparse data are implemented efficiently in LIBLINEAR [9], which underlies scikit-learn's LinearSVC. Because an SVM outputs a distance from the boundary rather than a probability, Platt's sigmoid fitting [10] is used to calibrate it so that its output can be averaged with Naive Bayes.

2.1.4 Algorithmic hiring and fairness

The hiring literature adds a constraint that the classification literature does not. Raghavan et al. found limited public information on how vendors validate bias-mitigation claims [1], and the 2018 withdrawal of a recruiting tool that penalised women's resumes [2] shows how a

model trained on historical data can reproduce historical bias. ResumeIQ responds modestly: identifiers (name, e-mail, phone, links) are removed before scoring, every score is decomposed into visible components, and the output is presented as a decision aid.

2.2 Summary Table

Ref.	Approach / Model	Setting	Reported Result
[3]	Inverse document frequency term weighting	Document-retrieval experiments	Terms occurring in fewer documents are better discriminators; IDF weighting improved retrieval
[4]	Comparison of term-weighting schemes	Several standard test collections	Schemes combining term frequency, IDF and length normalisation performed well
[6]	Naive Bayes event models (multinomial vs. Bernoulli)	Text-classification corpora	Multinomial model generally better with large vocabularies
[7]	Support-vector network (soft-margin SVM)	Pattern-recognition benchmarks	Introduced the soft-margin SVM with strong benchmark results
[8]	SVM for text categorisation	Reuters and Ohsumed collections	SVMs outperformed k-NN, Naive Bayes, Rocchio and C4.5 in the reported experiments
[10]	Sigmoid calibration of SVM outputs	Support-vector classifiers	Posterior probabilities obtained from SVM scores via a fitted sigmoid
[1]	Review of algorithmic-hiring bias claims	Vendor practices (qualitative)	Limited public information on how bias mitigation is validated

Table 2.1 — Summary of Related Approaches

2.3 What This Told Us

The literature pointed the team to the same starting point: TF-IDF features with Naive Bayes and a linear SVM, the best-understood classifiers for sparse text (Iteration 1, Chapter 4). Because the two rest on different assumptions (generative and independence-based versus margin-based), the team kept both and averaged their calibrated probabilities in Iteration 2. The skill taxonomy, the five-part score, the role-affinity measure and name-blind scoring were the team's own additions, motivated by the explainability and fairness concerns in [1] and [2].

CHAPTER 3

PROJECT PLANNING AND TEAM ORGANISATION

3.1 Weekly PBL Progress Log

The project ran across the full PBL cycle, from problem framing at the Zeroth Review (21 June 2026) through the final web-application build and testing. The table below summarises milestones and the work completed; the Mentor Remarks column is reserved for the mentor's own comments and sign-off.

Week	Milestone / Task	Work Done	Mentor Remarks
1–2	Problem framing (Zeroth Review)	Selected the problem; fixed the input formats, the twelve job roles and the objectives; presented scope and aim.	
3–4	Concept exploration	Surveyed TF-IDF, Naive Bayes, SVM and algorithmic-hiring literature (Chapter 2); designed the skill taxonomy and role catalog.	
5–6	Iteration 1: baseline	Built text extraction and a first data generator; trained TF-IDF + Naive Bayes + SVM; 100% accuracy showed the data were too easy.	
7–8	Iteration 2: refinement	Noisier generator; rule-based parser, PII removal and five-part scoring; role-affinity measure; role recommendation.	
9–10	First web version	Flask app and JSON API with screening, job-match and insight pages; first 34 tests; Docker image and Render blueprint.	
11–12	Iteration 3: platform, report, demo	Accounts, PostgreSQL persistence and migrations, dashboard, shortlist management and security hardening; 78 tests on two databases; browser verification; final report.	

Table 3.1 — Weekly PBL Progress Log

3.2 Requirements

ResumeIQ operates on text extracted from uploaded resumes. The classifiers are trained on 2,400 generated resumes; a command-line option (--csv) retrains the same pipeline on any CSV of labelled resumes, so no code change is needed to move from synthetic to real data.

All libraries are open source and the pinned versions below were used for every result in this report [11]–[14], [19].

Category	Requirement
Processor / RAM	Any x86-64 processor; 2 GB RAM or more. The running web worker, with the model and database loaded, used about 245 MB of resident memory (measured); no GPU is required
Programming language	Python 3.12 (Docker image); Python 3.13 for local development; HTML, CSS and JavaScript (user interface)
Libraries / frameworks	scikit-learn 1.9.1, NumPy 2.5.3, pandas 3.0.5, joblib 1.6.0 (machine learning); Flask 3.1.3, Gunicorn 26.2.0, Flask-Login, Flask-WTF, Flask-Limiter (web and accounts); SQLAlchemy 2.1.3, Alembic 1.20.0 (database); pypdf 6.17.0, python-docx 1.2.0, olefile 0.47 (parsing)
Database	SQLite (development, tests); PostgreSQL 16 (production, tests); Alembic migrations [19]
Tools	antiword (legacy .doc files); VS Code; pytest 9; Chromium (Playwright) for browser checks; Git and GitHub with a GitHub Actions workflow
Deployment	Docker container served by Gunicorn on Render with a managed PostgreSQL database (render.yaml blueprint) [15]

Table 3.2 — Hardware and Software Requirements

3.3 Feasibility

Training both classifiers, including 5-fold cross-validation, takes about 9 seconds on a single CPU core, and the trained model file is under 3 MB. Screening all fourteen sample resumes (four file formats, including text extraction and saving the results to the database) took a median of 0.19 s and recommending roles for one resume took about 16 ms, so the complete PBL cycle (baseline, refinement and final system) was achievable within the twelve-week timeframe on ordinary student laptop hardware.

CHAPTER 4

ITERATIVE DESIGN AND DEVELOPMENT

4.1 System Architecture

A recruiter's browser talks over HTTPS to a Flask application served by Gunicorn inside a Docker container. The application has four parts: a security layer (response headers, CSRF checks, rate limits), an authentication blueprint, the page views and a JSON API. Behind them sit the machine-learning engine, loaded once at start-up from the trained model file, and a relational database that holds users, screenings and candidates (Figure 4.1). Section 4.4 describes the ML pipeline in detail: resume → text extraction → NLP parsing → normalisation → TF-IDF → Naive Bayes and SVM → five-part score → ranked shortlist.

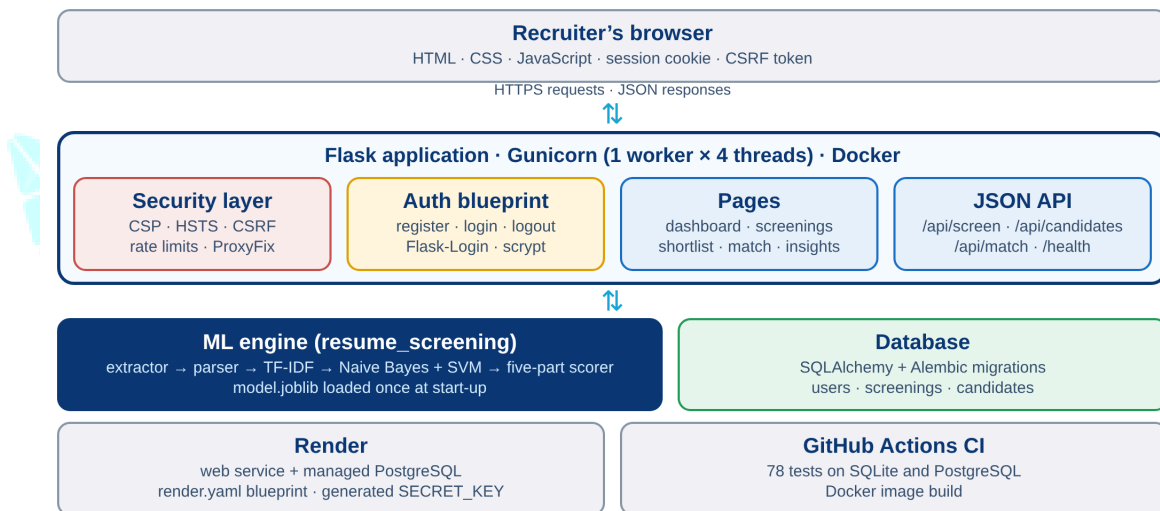


Figure 4.1 — System Architecture Diagram

A resume is accepted as PDF, DOC, DOCX or TXT (up to 30 files of 5 MB each per run); the file type is checked by extension and content signature, text is extracted with pypdf, python-docx or antiword, and a file with no readable text (for example a scanned image) is rejected with a clear message. The parser then recognises skills, education, experience and contact details, removes the personal identifiers for scoring and maps each skill to one canonical token.

The TF-IDF vectoriser feeds a Multinomial Naive Bayes model and a calibrated linear SVM whose probabilities are averaged; the scoring engine combines them with cosine similarity, skill coverage, experience and education into a 0–100 score, and the result is saved against the logged-in user. Offline, the training script writes model.joblib and metrics.json.

4.2 Iteration 1: Baseline

The simplest working version generated resumes from the role catalog using role-specific experience-bullet templates, vectorised them with TF-IDF (unigrams and bigrams) and trained a Multinomial Naive Bayes classifier and a calibrated linear SVM on an 80/20 stratified split. All three predictors (the two models and their average) scored 1.000 accuracy on the hold-out set and 1.000 in 5-fold cross-validation.

A perfect score on a twelve-class problem was a warning sign rather than a success. Inspection showed that each role's bullet templates were unique to that role, so the label could be recovered from a handful of template phrases; the model was memorising the generator, not learning to separate roles. This finding motivated Iteration 2.

4.3 Iteration 2: Refinement

Iteration 2 made the data harder and the system more complete. The generator was changed so that resumes could no longer be identified by template phrases:

- 28% of experience bullets are generic (for example “participated in code reviews and daily stand-ups”) and carry no role information; 14% are borrowed from a neighbouring role, and 35% for hybrid resumes.
- 25% of resumes are hybrids that mix the skills of a neighbouring role; 12% use a job title from a neighbouring role; 20% are sparse resumes listing only three to six skills.
- Each candidate lists a random 35–90% of the role's core skills, so some resumes are only weakly characteristic of their role.

With these changes the hold-out accuracy moved to 0.996 (Naive Bayes), 0.992 (SVM) and 0.996 (ensemble), and after the per-candidate skill-rate variation and a larger hybrid share it settled at 0.998 for all three. The data remain highly separable, which is a property of synthetic data, and this is stated as a limitation in Chapter 6 rather than tuned away. As a more meaningful check, five resumes written by hand in a different style from the generator were added to the test suite; all five were classified to the correct role.

Iteration 2 also improved the scoring. The first version measured role fit as the cosine similarity between the resume's and the job's role-probability vectors. Because the classifiers are confident, these vectors are almost one-hot, so a resume for a closely related role scored essentially zero: for the Data Scientist example job, the Machine Learning Engineer sample scored 0.004. Replacing the cosine with a role-affinity measure (full credit for the same role

and 0.4 for neighbouring roles) raised it to 0.395 while keeping the Data Scientist sample at 0.983. The grade thresholds were recalibrated from 75/55/35 to 70/50/30 after observing that clearly well-matched resumes scored between the mid-sixties and the low seventies, and name-blind scoring was verified by a test showing that changing a resume's name and e-mail leaves its score unchanged.

4.4 Final Approach

The final system is a two-classifier text pipeline with a transparent scoring layer. Its components are:

- TF-IDF features: unigrams and bigrams, minimum document frequency 2, maximum 0.9, English stop-words removed, sublinear term frequency, L2 normalisation, 11,688 features after fitting. With smoothed inverse document frequency, the weight of term t in document d is shown below, and each vector is then length-normalised so that the dot product of two vectors equals their cosine similarity [3]–[5].
- Multinomial Naive Bayes (smoothing $\alpha = 0.05$) [6]: the class posterior is proportional to the class prior times the product of the word likelihoods raised to their counts.
- Linear SVM (LinearSVC, $C = 0.8$) [7]–[9], calibrated with a sigmoid ($cv = 3$) [10] so that it outputs probabilities.
- Ensemble: the average of the Naive Bayes and calibrated SVM probabilities gives the role distribution of a document.

$$w_{t,d} = (1 + \ln tf_{t,d}) \times (\ln[(1 + N) / (1 + df_t)] + 1)$$

$$P(c | d) \propto P(c) \prod_t P(t | c)^{tf_{t,d}} \quad \cos(a, b) = a \cdot b / (\|a\| \|b\|)$$

The match score of a resume against a job description is a weighted sum of five components, each on a 0–1 scale. When the job description states no experience or education requirement, that component is dropped and the remaining weights are re-normalised (Table 4.1).

$$Score = 100 \times \sum_k w_k c_k / \sum_k w_k \quad (\text{sum over the components available for the job})$$

Component	Weight	How it is computed
Text similarity	40%	TF-IDF cosine similarity between the name-blind resume and the job description, divided by a ceiling of 0.45 and capped at 1
Skill coverage	30%	Share of the job's technical skills found in the resume; a related skill from the same family (for example MySQL for PostgreSQL) earns half credit

Component	Weight	How it is computed
Role fit	15%	$\text{Presume}^T A p_{\text{job}}$, where p are role distributions and A is the role-affinity matrix (1 for the same role, 0.4 for neighbours)
Experience	10%	$\min(1, \text{estimated years} / \text{years required})$; years are the union of dated ranges in the experience section or the stated figure, whichever is larger
Education	5%	1 if the highest degree meets the requirement, otherwise degree level / required level (1 = school ... 5 = doctorate)

Table 4.1 — Components of the Match Score

Scores are graded Strong (70 or above), Good (50–69), Partial (30–49) and Weak (below 30), and the shortlist is the top N resumes by score. Besides ranking, the engine recommends roles for a single resume by combining text similarity to each role profile (30%), weighted skill fit to the role's core and secondary skills (35%) and the classifier probability (35%), and lists the core skills the resume lacks as skills to learn.

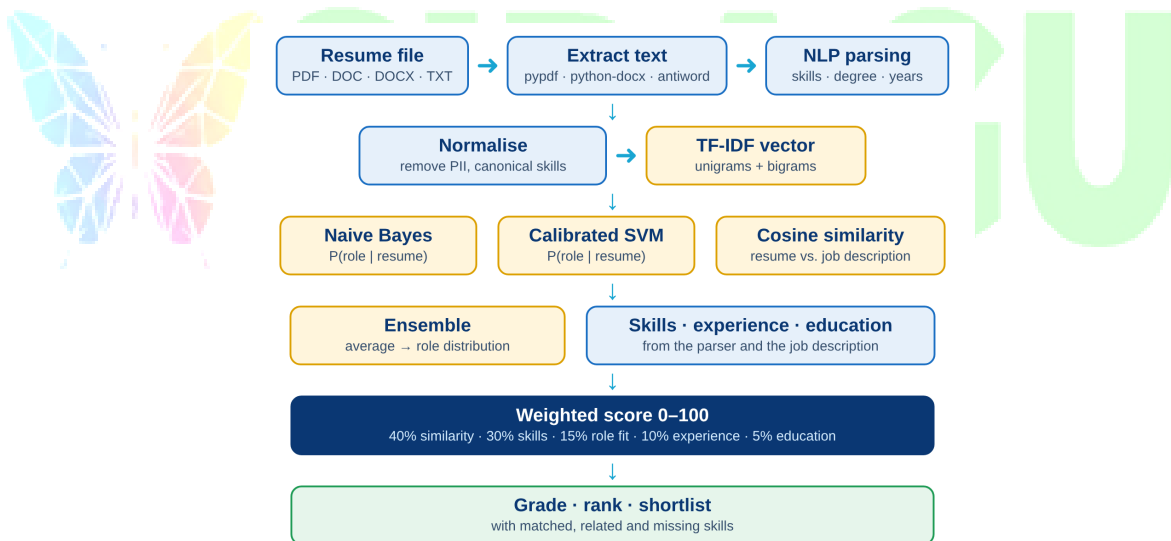


Figure 4.2 — ML Scoring Pipeline Data Flow

4.5 Training Procedure

The labelled data are split 80/20 with stratification (1,920 training and 480 test resumes, 40 test resumes per role). Five-fold stratified cross-validation is run on the training portion with the TF-IDF vectoriser re-fitted inside every fold, so no vocabulary or document-frequency information leaks from the validation fold. The final vectoriser and classifiers are then fitted on the training split and evaluated once on the untouched test split; the model shipped with the application is exactly the model that the reported hold-out metrics describe.

Component	Configuration
Dataset	2,400 synthetic resumes, 12 roles $\times$ 200, generator seed 42
Split	80/20 stratified: 1,920 training / 480 test; 5-fold stratified cross-validation on the training split
Text normalisation	Contact details removed; skills replaced by canonical tokens; lower-cased; non-alphanumeric characters removed
TF-IDF	ngram (1, 2), min_df 2, max_df 0.9, sublinear tf, English stop-words, max 30,000 features (11,688 used)
Naive Bayes	MultinomialNB, alpha = 0.05
SVM	LinearSVC (C = 0.8, random_state 42) wrapped in CalibratedClassifierCV (sigmoid, cv = 3)
Ensemble	Mean of the two predicted probability vectors; arg-max gives the predicted role
Output	model.joblib (vectoriser, both classifiers, labels) and metrics.json

Table 4.2 — Dataset and Model Configuration

4.6 Iteration 3: Multi-user Platform

The first web version was open to anyone and kept nothing, which suits a demonstration but not a recruiter who wants to return to yesterday's shortlist. Iteration 3 therefore added accounts, persistence and shortlist management without changing the ML pipeline: the stored results are exactly the engine's output, and a test asserts this. The data model has three tables (Table 4.3). Resume files themselves are never stored.

Table	Main columns	Purpose and constraints
users	email (unique), name, password_hash, session_token, failed_logins, locked_until	One row per recruiter; e-mail is lower-cased; the password is stored only as a script hash
screenings	user_id, job_title, job_description, job_info, top_n, total, errors, created_at	One screening run; deleting a user deletes the runs (ON DELETE CASCADE)
candidates	screening_id, user_id, rank, name, email, score, grade, recommended, status, notes, data (JSON)	One scored resume; status is pending, shortlisted or rejected; indexed on user, screening, score and status

Table 4.3 — Database Tables (SQLAlchemy models, created by an Alembic migration)

Security was designed in rather than added afterwards. The OWASP verification standard [16] guided the checklist, script [17] the password storage and a Content Security Policy [18]

the browser restrictions; Table 4.4 lists the controls, each covered by an automated test.

Area	Control in ResumeIQ
Passwords	At least 8 characters with a letter and a number; stored as script hashes (Werkzeug default, N = 32768, r = 8, p = 1); a check costs about 108 ms
Login	Unknown e-mail and wrong password give the same message and timing; 5 failures lock the account for 15 minutes; 15 attempts per minute per address; post-login redirects limited to same-site paths
Sessions	HttpOnly, SameSite=Lax and (in production) Secure cookies; a per-user session token that is rotated on password change so all other sessions stop working
Requests	CSRF token on every form and every state-changing API call; screening limited to 60 and job matching to 120 requests per hour per user; 30 sign-ups per hour per address
Data access	Every query is filtered by the logged-in user; another user's screening or candidate returns 404; CSV export neutralises spreadsheet formulas
Browser	Strict Content Security Policy (no inline scripts or styles), HSTS, same-origin referrer policy and no-store caching of account pages
Secrets and deployment	SECRET_KEY and DATABASE_URL come only from environment variables; production refuses to start without a secret key and, on Render, without a database

Table 4.4 — Security Controls

CHAPTER 5

IMPLEMENTATION

5.1 Module Description

Machine-learning package: resume_screening/

- extractor.py: converts PDF (pypdf), DOCX (python-docx), DOC (antiword, with an olefile fallback) and TXT files to text, checks file signatures and raises user-readable errors for empty, encrypted or scanned files.
- skills.py and parser.py: the 164-skill taxonomy with aliases and related-skill families, section detection, contact-detail extraction, experience estimation (union of dated ranges), education-level detection and job-description parsing.
- text_utils.py, catalog.py and dataset.py: removal of personal identifiers, canonical skill tokens, the twelve job-role profiles and the synthetic resume generator.
- train.py and engine.py: train and evaluate Naive Bayes and the calibrated SVM and write the model and metrics; load the model, rank resumes against a job description, compute the five-part score and recommend roles.

Web platform: app.py and webapp/

- app.py: the application factory create_app(), which loads the configuration, initialises the extensions, registers the blueprints, error handlers and security headers, and exposes the WSGI application used by Gunicorn.
- settings.py and extensions.py: environment-driven configuration (SECRET_KEY, DATABASE_URL, cookie flags, production safeguards) and the shared SQLAlchemy, Alembic, Flask-Login, CSRF and rate-limiter objects.
- models.py and migrations/: the User, Screening and Candidate models and the Alembic revision that creates them; forms.py: validated registration, login, password-change and account-deletion forms.
- auth.py: sign-up, login, logout, lockout and account settings; views.py: the dashboard, screening list, screening detail, shortlist, job-match and insight pages; api.py: the JSON endpoints for screening, candidates, CSV export and job matching.
- templates/ and static/: the interface in HTML, CSS and JavaScript (dynamic content is built with textContent so uploaded text cannot inject markup); tests/: 78 pytest tests;

samples/: 14 fictional resumes; Dockerfile, start.sh, render.yaml and .github/workflows/ci.yml for deployment and CI.

5.2 Key Code Snippets

The normalisation function removes personal identifiers and turns each skill spelling variant into one token, so “sklearn” and “scikit-learn” become the same feature:

```
def normalize_for_model(text: str) -> str:
    """Text fed to the TF-IDF vectoriser.

    Contact details are removed, skills are collapsed to canonical tokens
    (`scikit-learn` / `sklearn` -> `skill_scikit_learn`) so that spelling
    variants share one feature, and everything is lower-cased.
    """
    text = strip_contact_info(text)
    text = canonicalise_skills(text)
    text = text.lower()
    text = _TOKEN_CLEAN.sub(" ", text)
    return re.sub(r"\s+", " ", text).strip()
```

Code 5.1 — Name-blind text normalisation (text_utils.py)

On the account side, a failed login never reveals whether the e-mail exists, and repeated failures lock the account:

```
# Unknown e-mail and wrong password look identical (message and timing).
if user:
    user.failed_logins += 1
    if user.failed_logins >= current_app.config["MAX_FAILED_LOGINS"]:
        user.locked_until = now + timedelta(minutes=current_app.config["LOCKOUT_MINUTES"])
        user.failed_logins = 0
        log.warning("Account locked after repeated failed logins: user_id=%s", user.id)
        db.session.commit()
else:
    User.burn_password_check(form.password.data)
```

Code 5.2 — Failed-login handling and lockout (auth.py, excerpt)

5.3 User Interface / Demo

A new visitor sees the sign-up and login pages; the password field shows a strength meter and a Show toggle. After logging in, the recruiter lands on a dashboard with live totals (screenings, candidates screened, shortlisted, average score), the most recent screenings, the shortlist pipeline and the distribution of match grades. A sidebar gives access to New screening, Screenings, Shortlist, Job match and Model insights, and collapses to a menu on a phone.

To screen candidates, the recruiter pastes a job description or loads one of the twelve examples, drops resumes onto the upload area (or ticks the 14 built-in samples) and chooses

how many candidates the AI should recommend. The resumes are scored by the same engine as before and the run is saved to the account.

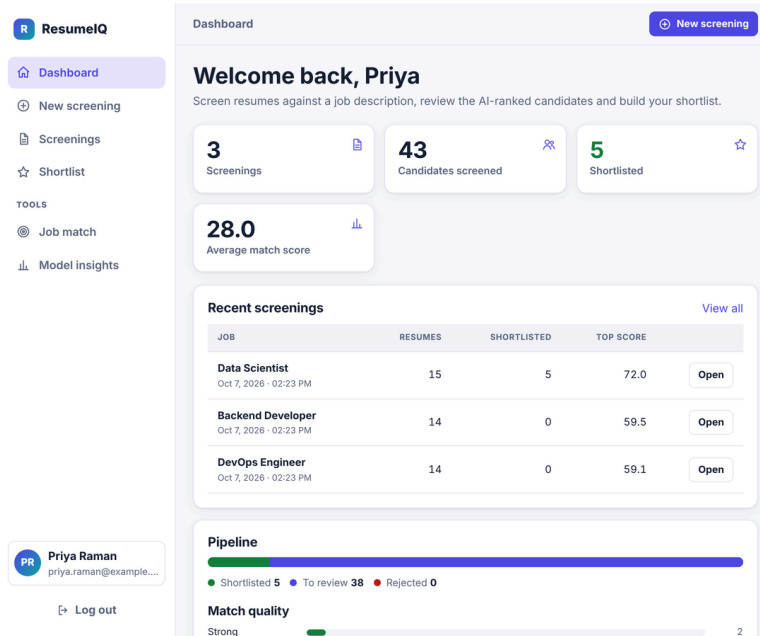


Figure 5.1 — Dashboard After Logging In

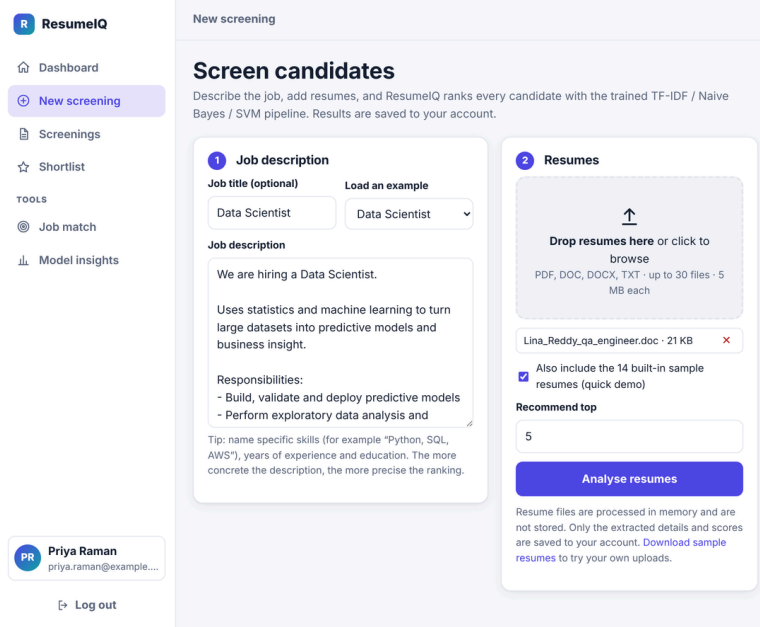


Figure 5.2 — New Screening Page with a Job Description and Resumes

The screening page shows the job type detected from the description, the skills extracted from it and counts per status. Candidates are listed in rank order as a table or as cards, with score, grade, experience, education and key skills, and can be searched, filtered by grade or status and sorted. Selecting a candidate opens a panel with the five score components, the

Naive Bayes and SVM predictions and the matched, related and missing skills, with buttons to shortlist, reject or reset the candidate and a notes field. “Shortlist AI top N” accepts the recommendation in one click.

ResumelQ

Dashboard
New screening
Screenings
Shortlist
Tools
Job match
Model insights

Screenings / Data Scientist

Data Scientist
Oct 7, 2026 · 02:23 PM · 15 resumes analysed · AI recommends the top 5

Shortlist AI top 5 Export CSV Delete

Detected role: Data Scientist (90%) 2+ years experience · Bachelor's required

SKILLS EXTRACTED FROM THE JOB DESCRIPTION
Statistics Machine Learning Data Analysis Feature Engineering Python Scikit-learn Pandas
SQL Deep Learning NumPy Data Visualization Jupyter TensorFlow

Search name, skill, email or role...

All matches Score: high to low

Table Cards

All 15 To review 10 Shortlisted 5 Rejected 0

RANK	CANDIDATE	MATCH SCORE	EXPERIENCE	EDUCATION	KEY SKILLS
#1	David Sato david.sato@example.com	72.0 Strong	3 yrs	Master's	Statistics Machine Learning +6
#2	James Mensah james.mensah@example.com	70.7 Strong	3 yrs	Bachelor's	Statistics Machine Learning +6

Shortlisted the AI's top 5 candidates

Priya Raman
Log out

Figure 5.3 — Saved Screening: 15 Resumes Ranked, AI Top 5 Shortlisted

The Shortlist page gathers the shortlisted candidates of every screening, with search, filters, notes and CSV export. The Job match page recommends roles for one resume, and Model insights reports the dataset, the model comparison and the confusion matrix and states that the data are synthetic.

ResumelQ

Dashboard
New screening
Screenings
Shortlist
Tools
Job match
Model insights

Shortlist

Shortlisted candidates
Everyone you've shortlisted across all screenings. Add notes, track decisions, and export the list.

All screenings Export CSV

Search name, skill, email or role...

All matches Score: high to low

Table Cards

All 43 To review 38 Shortlisted 5 Rejected 0

RANK	CANDIDATE	MATCH SCORE	EXPERIENCE	EDUCATION	KEY SKILLS
#1	David Sato david.sato@example.com Data Scientist	72.0 Strong	3 yrs	Master's	Statistics Machine Learning +6
#2	James Mensah james.mensah@example.com Data Scientist	70.7 Strong	3 yrs	Bachelor's	Statistics Machine Learning +6
#3	Isabella Nair isabella.nair78@example.com Data Scientist	46.6 Partial	6.8 yrs	Bachelor's	Statistics Machine Learning +4

Priya Raman

Figure 5.4 — Shortlist Page Across All Screenings

5.4 Interfaces and Deployment

The same engine serves the web pages and a JSON API, so the system can be used from a browser or called by another program after logging in. Table 5.1 lists the main endpoints. Uploads are accepted as multipart form data and errors are returned as JSON with a plain-language message.

Endpoint	Method	Purpose
/register, /login, /logout, /account	GET, POST	Sign up, log in, log out, change password or delete the account (rate-limited, CSRF-protected)
/dashboard, /screenings, /screenings/new, /shortlist	GET	Dashboard, saved screenings, new screening form and the cross-screening shortlist
/screenings/<id>	GET	One saved screening with its ranked candidates (owner only)
/api/screen	POST	Rank uploaded resumes (up to 30) against a job description, save the run and return its URL
/api/screenings[/id]	GET, DELETE	List, open or delete the user's screenings
/api/candidates[/id/bulk/export.csv]	GET, PATCH, POST	Search, filter and page candidates; shortlist, reject, annotate or bulk-update them; export CSV
/api/match, /api/roles, /api/samples, /api/model	POST, GET	Role recommendations for one resume, example job descriptions, sample resumes and training metrics
/health	GET	Health check that also confirms the database is reachable

Table 5.1 — Web Pages and JSON API Endpoints

Deployment follows the standard container route. The Dockerfile starts from a slim Python 3.12 image, installs antiword for legacy .doc files and the pinned requirements, and trains the model while the image is built, so the deployed model is exactly the one that was evaluated. At start-up, start.sh checks the configuration, applies the Alembic migrations (retrying while the database wakes up) and launches Gunicorn with one worker and four threads, which keeps the resident memory at about 245 MB (measured) inside the 512 MB limit of Render's free plan. The render.yaml blueprint creates the web service and a PostgreSQL database, generates SECRET_KEY, injects DATABASE_URL and sets REQUIRE_PERSISTENT_DB

so that the site refuses to run on a throw-away database. A GitHub Actions workflow runs the tests on SQLite and PostgreSQL and builds the image, boots it against a throw-away PostgreSQL and checks /health on every push.

Test file	Tests	What it checks
test_skills_parser.py	12	Skill aliases and ambiguous words, text normalisation, contact details, experience from overlapping dates, education levels, job-description parsing
test_extractor.py	9	TXT, DOCX, PDF and legacy DOC extraction, and friendly errors for empty, corrupt, scanned or unsupported files
test_engine.py	6	Metrics consistency, five hand-written resumes, ranking order, name-blind scoring, re-weighting, role recommendations
test_app.py	11	Page rendering, security headers, no inline scripts or styles, health check, job match, restricted sample downloads, input validation, error pages
test_auth.py	21	Registration rules, hashed passwords, login and logout, identical failure messages, lockout, safe redirects, password change rotating sessions, account deletion, CSRF, rate limits, cookie flags
test_screenings.py	13	Stored results equal engine output, shortlist, reject and notes, bulk updates, search, filter and paging, CSV export, deletion, isolation between users, dashboard counts
test_config.py	6	Database URL normalisation, SECRET_KEY and persistent-database requirements in production, migrations build the same schema as the models

Table 5.2 — Automated Test Suite (78 tests)

Besides the automated tests, the platform was driven in a real browser (Chromium) over HTTPS against PostgreSQL 16: an account was registered, resumes were screened, candidates were shortlisted, the CSV export was downloaded and the pages were checked at desktop and phone width with the console free of script and Content-Security-Policy errors. Start-up was also checked in production mode with a missing secret key, a missing database and a database fallback.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Evaluation Metrics

Role classification is a supervised problem, so it is evaluated with the standard classification metrics on the 480 held-out resumes: accuracy (share of resumes assigned the correct role), precision and recall (macro-averaged over the twelve roles) and the F1-score, the harmonic mean of precision and recall. Five-fold cross-validation on the training split gives a second, independent estimate of accuracy. Ranking quality cannot be measured against ground truth because no human ranking of the resumes exists, so it was examined qualitatively: whether resumes of the job's role rise to the top, and whether scores respond sensibly to missing skills, experience and education.

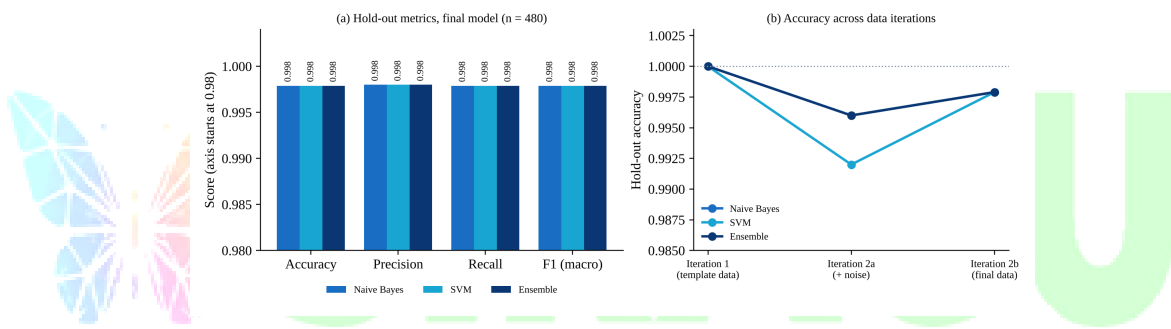


Figure 6.1 — Evaluation Metrics: Final Model and Accuracy Across Data Iterations

Three further instruments were used: (a) a set of five hand-written resumes, in a style different from the generator, that must be classified to the correct role; (b) behavioural tests of the scoring engine (ranking order, name-blindness, re-weighting when a requirement is missing); and (c) 78 automated tests covering the parser, extractor, engine, accounts, persistence and web interface, plus verification of the pages in a real browser at desktop and phone width.

6.2 Results Across Iterations

Version	Training data	Hold-out accuracy (NB / SVM / Ensemble)	Key finding
Iteration 1	Template resumes only	1.000 / 1.000 / 1.000	Perfect score; label recoverable from template phrases, so the data were too easy

Version	Training data	Hold-out accuracy (NB / SVM / Ensemble)	Key finding
Iteration 2a	+ generic and borrowed bullets, hybrids, sparse resumes	0.996 / 0.992 / 0.996	Realistic noise lowered accuracy slightly; 5-fold CV 0.997 / 0.996
Final (2b)	+ per-candidate skill-rate variation, 25% hybrids	0.998 / 0.998 / 0.998	5-fold CV 0.997 / 0.994; hand-written resumes 5/5 correct

Table 6.1 — Model Evaluation Results Across Iterations

On the final model, the Naive Bayes classifier, the SVM and their ensemble each reached 99.8% accuracy (479 of 480 test resumes), with macro precision, recall and F1 of 0.998. The only error is one Cybersecurity Analyst resume classified as a DevOps Engineer, a neighbouring role in the catalog that shares Linux, monitoring and networking skills (Figure 6.2).

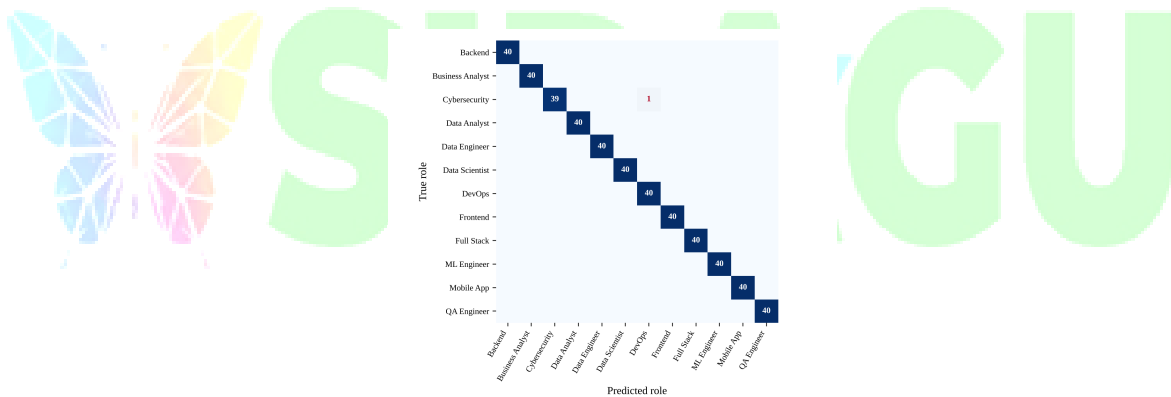


Figure 6.2 — Confusion Matrix of the Ensemble on the Hold-out Set

Job role	Precision	Recall	F1-score	Test resumes
Backend Developer	1.000	1.000	1.000	40
Business Analyst	1.000	1.000	1.000	40
Cybersecurity Analyst	1.000	0.975	0.987	40
Data Analyst	1.000	1.000	1.000	40
Data Engineer	1.000	1.000	1.000	40
Data Scientist	1.000	1.000	1.000	40
DevOps Engineer	0.976	1.000	0.988	40
Frontend Developer	1.000	1.000	1.000	40
Full Stack Developer	1.000	1.000	1.000	40

Job role	Precision	Recall	F1-score	Test resumes
Machine Learning Engineer	1.000	1.000	1.000	40
Mobile App Developer	1.000	1.000	1.000	40
QA Engineer	1.000	1.000	1.000	40

Table 6.2 — Per-role Results of the Ensemble on the Hold-out Set

To see the ranking behave end to end, the fourteen built-in sample resumes were screened against the example Data Scientist job description (a median of 0.19 s for all fourteen, including text extraction from PDF, DOCX, DOC and TXT files and saving the run). Figure 6.3 and Table 6.3 show the highest-ranked candidates.

Rank	Candidate	Predicted role	Score	Grade	Text sim.	Skills
1	David Sato	Data Scientist	72.0	Strong	54	69
2	James Mensah	Data Scientist	70.7	Strong	48	73
3	Isabella Nair	Data Analyst	46.6	Partial	18	62
4	Sneha Mehta	Machine Learning Engineer	40.7	Partial	9	54
5	Ingrid Rossi	Data Engineer	35.5	Partial	11	54
6	Lina Muller	Full Stack Developer	23.1	Weak	3	23
7	Kwame Patel	Business Analyst	20.8	Weak	3	15
8	Aiko Lewis	Frontend Developer	16.6	Weak	4	0

Table 6.3 — Top Candidates for the Data Scientist Job (14 Sample Resumes)

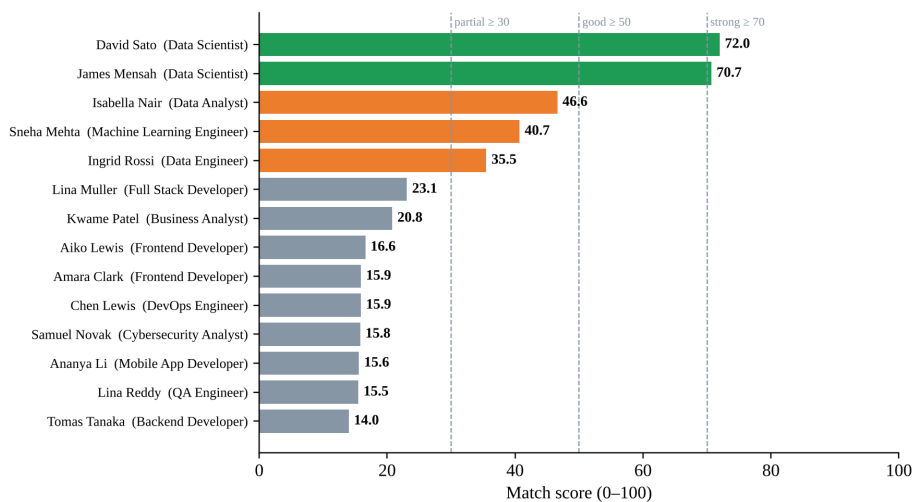


Figure 6.3 — Live Ranking of the Sample Resumes for the Data Scientist Job

6.3 Platform Verification and Performance

The platform was measured through the application's own test client on SQLite, so every call used the real views, database writes and ML engine (Table 6.4). The figures describe one laptop-class machine and are not a load test.

Measurement	Result
Screening 14 sample resumes (4 file formats), including extraction and saving	median 0.187 s, maximum 0.281 s
Opening a 14-candidate screening list (API)	median 5.7 ms
Loading the dashboard	median 5.0 ms
One password check (bcrypt)	median 108 ms (deliberately slow)
Resident memory of the running web process	about 245 MB
Automated tests	78 passed on SQLite and on PostgreSQL 16

Table 6.4 — Measured Performance and Verification of the Platform

6.4 Discussion

The most consistent observation is that the two classifiers agree: Naive Bayes and the calibrated SVM predicted the same role for every resume in the sample screening, and their one-error result on the hold-out set is shared. This is partly a property of the synthetic data, in which each role has a distinctive skill vocabulary; the ensemble's value would be expected to show on noisier real resumes, where the two models are likely to disagree more often. This was not tested.

The scoring layer, rather than the classifier, is what makes the output useful to a recruiter. In the Data Scientist example the two Data Scientist resumes scored in the strong band (72.0 and 70.7), related roles in the partial band, and unrelated roles below 30. Because every component is shown, a recruiter can see why a candidate ranked where they did, for instance a missing Deep Learning skill, and can override the score; the shortlist, reject and notes controls keep that human decision next to the AI recommendation. The related-skill rule and the role-affinity measure were introduced because the first, stricter versions penalised near-misses that a human reader would forgive.

Name-blind scoring was verified directly: replacing a resume's name and e-mail address leaves its score unchanged. This removes one channel for bias but is not a fairness guarantee,

because other fields such as institution names or employment gaps can still correlate with protected characteristics; a proper audit needs real, demographically annotated resumes that were not available to the team.

Several defects were found only because the system was tested as a web service rather than a notebook: inline style attributes blocked by the Content Security Policy silently removed spacing; the CSV spreadsheet-injection guard altered phone numbers beginning with “+”; a loading overlay stayed visible because a CSS display rule overrode the hidden attribute; choosing an example job description before the script had loaded did nothing; and a sign-up limit of 10 per hour proved too tight for a shared network. Each was corrected and covered by a test or a browser check.

6.5 Limitations

- The classifiers were trained on 2,400 synthetic resumes. The 99.8% hold-out accuracy shows that the pipeline works and the roles are separable in the generated data; it is not a real-world accuracy estimate, and real resumes are expected to give lower and more variable results. The five hand-written resumes are a modest sanity check, not a substitute for real evaluation.
- Extraction is rule-based over a 164-skill taxonomy; skills, phrasings and date formats outside the rules are missed, and a trained named-entity model would be more robust.
- Scanned or image-only resumes are rejected (no OCR), and unusual layouts such as multi-column designs can reduce extraction quality.
- The score weights, the similarity ceiling (0.45), the affinity value (0.4) and the grade thresholds are design choices calibrated on the sample resumes, not parameters learned from recruiter decisions.
- The system supports twelve roles and English text only and is a decision aid that requires human review.
- The accounts have no e-mail verification or password reset, use library-default script parameters, and count rate limits per worker, which suits the single-worker deployment but not a scaled-out one.
- Saved candidate records hold names, e-mails and phone numbers, so a real deployment needs a consent and retention policy; Render's free PostgreSQL plan also expires after about 30 days.

CHAPTER 7

TEAM REFLECTION AND LEARNING OUTCOMES

7.1 Individual Reflections

Sadhana Shanmugam: I focused on the machine-learning pipeline (extractor, skill taxonomy and parser, TF-IDF, Naive Bayes and SVM training, data generator, scoring engine) and, in the last iteration, the database layer. The biggest thing I learned is that a perfect score is a question, not an answer: when the first model reached 100% accuracy I had to discover that the generator, not the model, was doing the work. Moving from a stateless demo to saved screenings taught me a second lesson: storing the engine's output unchanged, and testing that it is unchanged, kept the ML and the platform independent, and the Alembic migration forced me to think about the schema before any data existed.

Dhanya Sri Mohandass: I focused on the interface, the account system, the automated tests and the deployment files. Building the interface with a strict Content Security Policy showed me how many small things can go wrong between a working model and a usable product, from blocked inline styles to a loading overlay that would not hide. Writing the login code taught me that security is mostly about the cases nobody demonstrates: identical messages for unknown users, lockout, rotating sessions on a password change and checking ownership on every query. The most valuable habit was driving the pages in a real browser, on PostgreSQL and at phone width, and writing a test for each defect once it had been found.

7.2 Team Learning

Splitting ownership along the pipeline/interface boundary worked well once the JSON response format of the engine was fixed early, because it allowed both members to work in parallel: the interface was built against a documented response shape while the scoring logic was still being refined underneath it. The same contract made Iteration 3 possible, since the database stores that response unchanged. What we would do differently is lock the response format even earlier; several interface components had to be reworked when the score breakdown and the related-skills field were added.

The change that most improved the project was the decision, after the first perfect score, to treat the evaluation itself as something to be tested. It led to a harder data generator, to a hand-written sanity check, to an honest limitations section, and to scoring components that a recruiter can read instead of a single opaque number. Both members also learned to review

each other's work before every milestone, which caught several defects before they reached the demonstration.

7.3 Course Outcomes — Evidence Summary

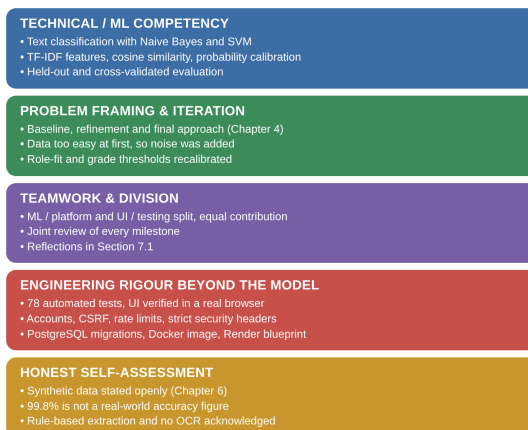


Figure 7.1 — Course Outcomes

- Technical / ML competency: the Naive Bayes and SVM text classifiers, TF-IDF features, probability calibration and cosine-similarity scoring, evaluated with a hold-out set and cross-validation, demonstrate applied understanding of supervised text classification (Chapters 2 and 4).
- Problem framing and iteration: the weekly progress log (Table 3.1) and the Iteration 1 → Iteration 2 → Final Approach progression (Chapter 4) show the build evolving in response to specific findings, notably the first perfect score, the one-hot role-fit result and the need for accounts once results had to be kept.
- Teamwork and division of labour: the Team Roles table and the reflections above (Section 7.1) show a consistent pipeline/platform and interface/security split with equal contribution and joint review before each milestone.
- Engineering rigour beyond the model: 78 automated tests on two databases, browser verification, defects found and fixed (Chapter 6), hashed passwords, CSRF protection, a strict Content Security Policy and a Docker and Render deployment show that the system was treated as a service to be verified, not only a model to be fitted.
- Honest self-assessment: Chapter 6 states plainly that the data are synthetic and that the headline accuracy is not a real-world estimate, which is the candid, reflective evaluation the PBL format asks for.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

ResumeIQ demonstrates that a classical machine-learning pipeline (TF-IDF features, a Multinomial Naive Bayes model and a calibrated linear SVM) combined with rule-based NLP extraction and a transparent five-part score can turn unstructured resumes into a ranked, explainable shortlist and into role recommendations, without using a candidate's name or contact details and on ordinary laptop hardware. On 480 held-out synthetic resumes the ensemble reached 99.8% accuracy, all five hand-written resumes were matched to the correct role, and fourteen sample resumes in four file formats were screened and saved in about 0.19 s. Delivered as a multi-user Flask web application with accounts, a PostgreSQL-backed history of screenings, shortlist management, 78 automated tests and a Docker and Render deployment package, the project addressed its driving question directly: text-based models can separate job roles and rank resumes in a way that a recruiter can inspect and act on. Because the training data are synthetic, the numerical results show that the pipeline works rather than how accurate it would be on real resumes.

8.2 Future Scope

- Train and validate on real, labelled resumes (the --csv option already retrains the same pipeline), with proper consent and anonymisation, and run a fairness audit on demographically annotated data.
- Replace the rule-based extractor with a trained named-entity model, add optical character recognition for scanned resumes, and extend the skill taxonomy and role catalog beyond twelve roles and English.
- Learn the score weights and the similarity ceiling from recruiter decisions and feedback instead of fixing them by hand.
- Strengthen the accounts with e-mail verification, password reset, stronger script or Argon2id parameters and a shared rate-limit store, and add organisations and roles so that a hiring team can share a shortlist.
- Add audit logging, a data-retention policy for stored candidate records, and monitoring on a paid Render database plan.

REFERENCES

- [1] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in Proc. ACM Conf. on Fairness, Accountability, and Transparency (FAT*), Barcelona, Spain, 2020, pp. 469–481.
- [2] J. Destin, "Amazon scraps secret AI recruiting tool that showed bias against women," Reuters, Oct. 10, 2018.
- [3] K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval," Journal of Documentation, vol. 28, no. 1, pp. 11–21, 1972.
- [4] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.
- [5] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge University Press, 2008.
- [6] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," in Proc. AAAI-98 Workshop on Learning for Text Categorization, 1998, pp. 41–48.
- [7] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.
- [8] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in Proc. European Conf. on Machine Learning (ECML), 1998, pp. 137–142.
- [9] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, "LIBLINEAR: A library for large linear classification," Journal of Machine Learning Research, vol. 9, pp. 1871–1874, 2008.
- [10] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers, Cambridge, MA, USA: MIT Press, 1999, pp. 61–74.
- [11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [12] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020.
- [13] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conf., 2010, pp. 51–56.
- [14] Pallets Projects, "Flask documentation." [Online]. Available: <https://flask.palletsprojects.com/> (accessed 2026).
- [15] Render, "Blueprint specification (render.yaml)." [Online]. Available: <https://render.com/docs/blueprint-spec> (accessed 2026).
- [16] OWASP Foundation, "OWASP Application Security Verification Standard (ASVS), version 4.0.3," 2021. [Online]. Available: <https://owasp.org/www-project-application-security-verification-standard/> (accessed 2026).
- [17] C. Percival and S. Josefsson, "The script password-based key derivation function," RFC 7914, Internet Engineering Task Force, Aug. 2016.
- [18] W3C, "Content Security Policy Level 3," W3C Working Draft. [Online]. Available: <https://www.w3.org/TR/CSP3/> (accessed 2026).
- [19] M. Bayer, "SQLAlchemy documentation" and "Alembic documentation." [Online]. Available: <https://docs.sqlalchemy.org/> and <https://alembic.sqlalchemy.org/> (accessed 2026).

APPENDIX

A.1 Full Source Code

[<https://github.com/sadhanashan16/Resumescreeing>]

A.2 Complete Weekly PBL Log

The full week-by-week log is summarised in Table 3.1 (Chapter 3); this build's log spans the twelve-week PBL cycle beginning at the Zeroth Review (21 June 2026). The Mentor Remarks column of Table 3.1 is left for the mentor to complete and sign.

A.3 Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
Sadhana Shanmugam	50%	50%	Built the extraction, NLP parsing, model training, scoring engine, data model and screening API.
Dhanya Sri Mohandass	50%	50%	Built the web interface, account and security layer, tests, sample data and deployment package.